

1. Evaluación de Cohesión y Acoplamiento

Para que un sistema sea de alta calidad, buscamos Alta Cohesión (que cada pieza haga una sola cosa muy bien) y Bajo Acoplamiento (que las piezas no dependan excesivamente entre sí).

- **Análisis de Separación:** Los componentes están correctamente segregados por capas. La decisión de separar la Capa de Lógica de la Capa de Datos es un acierto de calidad, ya que permite cambiar el motor de base de datos (ej. pasar de SQL Server a PostgreSQL) sin tener que reescribir la lógica de validación de archivos.
- **Evaluación de Cohesión:** Se observa una cohesión funcional alta. Por ejemplo, el componente Validación Binaria tiene una única responsabilidad: inspeccionar la integridad de los archivos. No se mezcla con la lógica de usuarios ni con el envío de correos, lo cual facilita enormemente las pruebas unitarias.
- **Análisis de Acoplamiento:** El acoplamiento es bajo y controlado. La comunicación se realiza mediante interfaces (como IValidar o ISave), lo que significa que un componente "llama" a un servicio sin necesidad de conocer cómo está implementado por dentro.

Hallazgo de QA: No se detectaron dependencias innecesarias directas entre la Capa de UI y la Capa de Datos, lo cual respeta el principio de "encapsulamiento de capas".

2. Identificación de Riesgos Arquitectónicos

Tras auditar el flujo de interacción y el diagrama XML, he identificado los siguientes puntos de dolor:

- **Riesgo de "Componente Dios" (Bloatware):** El componente Gestión de Expedientes corre el riesgo de volverse demasiado grande. Actualmente coordina la carga, la comunicación con la base de datos, el guardado en el Object Store y la validación. Si se le agregan más funciones, se volverá difícil de mantener.
- **Responsabilidades mal definidas en Alertas:** No queda claro si el componente Seguimiento y Alertas es meramente pasivo (guarda la alerta en la DB) o activo (envía correos o notificaciones push). Si es activo, requiere una dependencia externa a un servidor SMTP o servicio de terceros que no está modelada.
- **Inexistencia de dependencias circulares:** Afortunadamente, el flujo es unidireccional (hacia abajo). No hay casos donde la Base de Datos intente llamar a la Interfaz, lo cual es un indicador de buena salud arquitectónica.

3. Recomendaciones de Mejora

Para elevar el estándar de calidad del sistema, sugiero las siguientes acciones concretas:

1. **Implementar Procesamiento Asíncrono para Validaciones:** La dependencia entre Gestión de Expedientes y Validación Binaria debe ser asíncrona. Si un aspirante

sube 10 archivos pesados simultáneamente, la validación binaria podría saturar el servidor. Recomendando usar una cola de mensajes entre ambos.

2. Patrón Proxy para el Object Store: Dado que el acceso al almacenamiento externo puede ser lento o fallar, se recomienda envolver el componente Repositorio Object Store en un patrón de Circuit Breaker. Esto evitará que una caída de la nube bloquee todo el sistema de reclutamiento.
3. Fragmentación de la Gestión de Expedientes: Dividir el componente en dos: FileUploader (manejo de flujo de datos) y MetadataManager (gestión de información en SQL). Esto reducirá el riesgo de tener un componente demasiado grande.
4. Caché de Autorización en RBAC: El componente de Autenticación y RBAC es consultado en cada interacción de la Interfaz. Para evitar un cuello de botella en la Base de Datos SQL, se recomienda implementar una capa de caché (Redis o memoria local) para los permisos de los usuarios activos.
5. Interfaz de Visualización Independiente: Para cumplir con el RF-03 (visualizar sin descargar), el componente de Interfaz Web debería consumir una interfaz de "Streaming" directa del Object Store mediante tokens de acceso temporal (Presigned URLs), en lugar de pasar el archivo a través de la Capa de Lógica, ahorrando ancho de banda y memoria en el servidor.

Conclusión: El diseño es APROBADO con observaciones. La arquitectura modular es sólida y respeta los estándares UML. Si se aplican las recomendaciones de asincronía y manejo de fallos en el almacenamiento, el sistema tendrá una alta disponibilidad y facilidad de mantenimiento.